

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.

"hostname", "uptime_days", "managed_by_apic", "interfaces"

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null.

String (Cadena de texto)

Number (Número)

Boolean (Booleano)

Null (Nulo)

3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación? Selecciona las dos opciones correctas.

- A) El valor de 'vlan_id' debería estar entre comillas dobles obligatoriamente.
- B) Las claves como 'vlan_id' no deben llevar guiones bajos.
- C) Existe una coma final (trailing comma) después del valor de 'description'.
- D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red.

Evita caídas de red (*outages*): Un dato erróneo o mal formado puede desconfigurar los dispositivos masivamente y cortar la conectividad.

Previene estados inconsistentes: Impide que la configuración se aplique a medias y deje a los equipos en un estado impredecible.

Garantiza parámetros válidos: Asegura que los datos (IPs, VLANs, MTU) cumplan con las reglas de la red antes de ejecutarse.

Bloquea errores en la API: Permite rechazar la solicitud a nivel de software (error 400/422) sin poner en riesgo la infraestructura física.

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable `inventario_json` (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea 'down'.

```
# Imprimir todos los que estan en down
```

```
for d in inventario:
```

```
    if d["status"] == "down":
```

```
        print(d["hostname"])
```

Formato JSON	Formato XML
Pares clave-valor entre llaves ("clave": "valor")	Etiquetas jerárquicas de apertura y cierre (valor)
Ligero y compacto: no repite nombres de etiquetas	Más verboso: requiere etiquetas repetidas para cerrar cada campo
Fácil de leer y procesar por humanos y máquinas	Más recargado sintácticamente
Estándar preferido para REST APIs y aplicaciones modernas	Utilizado en sistemas heredados y protocolos como NETCONF